

生産・調達向け アプリ説明資料（ロジック準拠）

目的

- アプリの計算の考え方を前提に、意見を交換
-

1. 日替わり時刻（8時）

日替わり時刻を8:00としています。

- 1月19日 7:59 → 1月18日として扱う
- 1月19日 8:00 → 1月19日として扱う

この8時区切りは以下の処理に影響します：

- 実績の日付判定（工程作業入力の計画日）
 - 在庫計算の「今日」判定
 - 生産計画の開始日判定
-

2. アプリで扱う主要情報

在庫残量画面の項目名

- **内示:** 最終ラインでは顧客の内示 それ以外は後ラインの計画数
- **確定:** 最終ラインでは顧客確定の確定、それ以外は後ラインの実績数
- **計画:** 生産計画の数量
- **実績:** 実際に作れた数量
- **調整:** 棚卸差異などの補正
- **仕損:** 実績から差し引かれる不良数
- **在庫:** 現時点で持っている数量
- **計画在庫:** 計画に基づく将来の在庫見込み
- **進度:** 「本来その日までに作っておくべき量」との差

補足：月別合計の表示

- 画面上部で **予定（内示）** と **確定** の月別合計を表示
-

3. 在庫の概念（実在庫と計画在庫）

出庫の考え方

- 出庫は「後工程で消費された数量」を表す
- 出庫は「最終品」と「中間品」で計算方法が違う
 - **最終品:** 顧客確定（firm）を出庫として扱う
 - **中間品:** 後工程の数量 × BOM使用数 で出庫を計算

実在庫と計画在庫の出庫の違い

項目	出庫の計算
実在庫 (stock_qty)	後工程の 実績 (actual_qty) × BOM使用数
計画在庫 (planned_stock_qty)	過去は後工程の 実績 、今日以降は 計画値 × BOM使用数

- **実在庫**: 「今、物理的に何個あるか」を表すため、**実績**で出庫計算
- **計画在庫**: 過去は**実績**、今日以降は****計画値** (内示) **で出庫計算

実在庫

在庫 = 前日在庫 + 実績生産 - 実績出庫 (同日) + 調整 - 仕損

- **事実ベース**で同日に計上する

計画在庫

計画在庫 = 前日計画在庫 + 入庫 - 計画出庫 + 調整 - 仕損

- **過去**: 入庫=実績、出庫=最終品は確定/最終品以外は実績
- **今日以降**: 入庫=計画、出庫=計画値 (内示)

在庫計算の起点 (前々営業日ルール)

日替わり時刻 (8時) の関係で、在庫計算は**前々営業日の在庫**を起点とします。 (前々営業日 = カレンダー上で2営業日前、休日は飛ばす)

日付	処理
前々営業日以前	既存の在庫値をそのまま使用 (更新しない)
前営業日	前々営業日の在庫を起点に再計算
今日以降	前営業日の在庫を起点に再計算

理由: 7:59に実績入力 (前日扱い) → 8:01に在庫計算した場合、前日の在庫にはまだ7:59の実績が反映されていない可能性があるため。前々営業日は確定しているので、そこから再計算すれば正しい値になる。

4. 進度の概念 (顧客需要に対する遅れ/先行)

進捗とは

進捗は「顧客需要に対して実績が遅れているか先行しているか」を表す指標。 **ラインの計画とは関係なく**、顧客の需要 (内示/確定) に対する実績の状況を示す。

進度の計算式

$$\text{進度} = \text{前日進度} + \text{実績} - \text{需要 (LT遡り済み)} + \text{調整} - \text{仕損}$$

需要の取得（LineDemandから）

需要は **LineDemand**（受注からRoutingでLT遡りして展開済み）から取得：

優先順位	条件	使用する値
1	確定（firm_qty） > 0	firm_qty
2	確定なし、内示（forecast_qty） > 0	forecast_qty
3	どちらもなし	0

重要: 進度は**ラインの計画（plan_qty）とは無関係**。顧客需要に対する実績の遅れ/先行を表す。

リードタイム（LT）の優先順位

1. 工程ごとのリードタイム設定（RoutingStep.lead_time_days）
2. ラインの標準リードタイム（Line.lead_time_days）
3. 部品のリードタイム（BOMItem.lead_time_days）
4. 0（設定がない場合）

リードタイムの考え方（運用ルール）

- 基本は「ライン間は前日があれば良し」とする
- そのため、ライン間のリードタイムは **1日設定** を標準とする
- トラブルや欠員に備えてリードタイムを増やす運用は、計画在庫と進度の「実態」とズレるため、本アプリでは採用しない

LineDemand展開のLT換算ルール（最新）

- 展開はルーティングの **hierarchy_path** を辿り、**系統ごとに日付を計算**
- **DAY工程:** **lead_time_days** を営業日換算で遡る
- **MINUTE工程:** **duration_min** を累積し、営業日換算で日数に加算
 - 換算基準: 480分 = 1日
 - 切り上げ方式: **ceil(累積分 / 480)**
 - 例: 1日 + 20分 + 20分 → 1.166...日 → **2日**扱い

進度の具体例

例：LT=1日の場合

顧客需要：1/10に確定10個

↓ Routing展開でLT1日遡る

LineDemand：1/9に需要10個として展開される

1/9時点の実績が0個の場合:
 進度 = 0 - 10 = -10
 → 「1/9までに作るべき分が10個遅れている」

1/9時点の実績が15個の場合:
 進度 = 15 - 10 = +5
 → 「5個先行している」

進度の活用

- **進度がマイナス**: 後工程に影響を与える可能性あり → 対応が必要
- **進度がプラス**: 余裕あり → 他の遅れ品を優先しても良い

在庫・計画在庫・進度の比較

項目	需要/出庫の元データ	入庫の元データ	意味
在庫	後工程の 実績 (BOM経由)	実績	今ある物理的な数量
計画在庫	後工程の 計画値 (BOM経由)	計画	計画どおりの場合の在庫
進度	LineDemand (Routing展開済み)	実績	顧客需要に対する遅れ/先行

5. 生産計画の立て方（アプリの流れ）

画面: 生産計画入力

流れ

1. **取り込み**
 - 後工程計画から「需要」を算出して反映
 - 需要数量 (`demand_qty_plan`) として保存
2. **計画入力**
 - 計画数量と順序を入力
3. **保存**
 - 計画保存 → 工程展開 → 在庫・計画在庫再計算 → ガント生成
 - 進度は在庫残量画面の在庫再計算で再計算する

ルール

- 計画数量が0以下は削除扱い
- 順序は同日・同ラインで重複不可
- 複数ロットは「+」で追加

6. 工程展開の意味

- ライン最終品をBOMでライン内の工程に展開
- 展開結果は自ライン工程の計画として自動反映される

- つまり、ライン計画が工程の計画に自動変換される

7. 仕入れ計画の立て方（購買需要計算）

画面: 仕入れ計画入力

需要計算

1. 仕入先を選択
2. BOMItem (BUY/SUBCON) から購買部品を抽出
3. **親製品の計画数量を基準に需要を計算**
4. リードタイム分だけ営業日で前倒し
5. 需要として画面に反映

重要ルール

- **需要は「計画数量」を基準に計算**
 - 調達は「実際の計画数量」に基づく

実務手順

- 需要列を確認
- 計画列に発注数量を入力
- 生産計画変更時は **取り込みを再実行** する

仕入れ計画の表示項目

- **需要**: 後工程の計画数量から算出された購買需要 (**order_qty**)
- **実績**: 仕入れの実績入庫 (**actual_qty**)
- **在庫**: 実在庫 (**stock_qty**)
- **計画**: 計画発注数 (**plan_qty**)
- **計画在庫**: 計画在庫 (**planned_stock_qty**)
- **進捗**: 需要に対する実績の遅れ/先行 (**progress_qty**)

仕入れ進捗の考え方

仕入れの進捗は「購買需要に対して入庫実績が遅れているか先行しているか」を示す。
生産ラインの進捗ロジックを**仕入れ需要 (order_qty) **に置き換えて適用する。

進捗 = 前日進捗 + 実績 - 需要 + 調整 - 仕損

- **需要**: LineDemand (顧客需要をRoutingでLT遡りして展開した結果)
- **実績**: 仕入れの実績入庫 (**actual_qty**)
- **調整/仕損**: 仕入れでは通常 0 (運用で必要な場合のみ使用)

8. 全体の流れ（データフロー）



9. まとめ

- 在庫は「実在庫」と「計画在庫」で意味が違う
- 進度はLT廻りで計算し、遅れを可視化する
- 生産計画は **取り込み** → **入力** → **保存** が基本
- 仕入れ需要は **計画数量基準** + **LT前倒し** で算出する

補足: 一般的な生産システムとの違い

- 一般的には「顧客需要」をBOMとルーティングで全工程に展開する
- 本アプリは **後ラインの需要を基準に、自ラインへ展開** する
- 展開は **ライン単位で1ラインずつ** 行う
- そのため、**各ラインの計画を入力しないと前ラインの需要が確定しない**
- 顧客の内示に基づき、**最終ラインで長期計画を立てることが前提**
- 計画は **今日から設定日数以内がロック** となる（近い日付は変更不可、**日数は設定画面で変更可能**）

10. 需要日の決定ロジック（日替わり8時ルール適用）

前工程需要計算（社内計画）

前工程の需要日は、後工程の計画日を基準に決定します。

計画日の取得優先順位

優先順位	参照元	日付の取得方法
1	LineGanttPlan	<code>start_datetime</code> から日付を取得（日替わり8時ルール適用）

優先順位	参照元	日付の取得方法
2	LineBacklog	<code>plan_date</code> （フォールバック）

日替わり8時ルール適用

ガントの `start_datetime` から日付を取得する際、8時より前は前日扱いにします。

例：

`start_datetime = 2026-01-27 00:55:00`

↓ `0:55 < 8:00` なので前日扱い

業務上の日付 = 2026-01-26

中間品から最終品への検索

親製品（BOMの親）が中間品の場合、ガントには最終品しか登録されていないため、ルーティングから最終品を特定してガントを検索します。

例：

BOM親製品：294（中間品）→ ガントなし

↓ `RoutingStep` から最終品を特定

最終品：113（`is_line_final_product=True`）→ ガントあり

↓ 最終品のガントを使用

需要日の計算

前工程の需要日 = 後工程の計画日（日替わり8時ルール適用後） - リードタイム（営業日）

購買需要計算（仕入れ計画）

購買需要計算も同じロジックを適用します。

1. 親製品のLineBacklogから計画数量を取得
2. ガント（LineGanttPlan）があれば `start_datetime` から日付を取得
3. 中間品の場合は最終品のガントを検索
4. 日替わり8時ルールを適用して業務上の日付を決定
5. リードタイム分を営業日で前倒し

11. 計画保存時のデータ連動

保存フロー

1. 画面で「保存」
2. POST /line-plans/save → LinePlanに保存
3. POST /line-backlogs/expand_processes → 同一ライン内の工程に展開
4. POST /line-backlogs/recalculate_inventory → 在庫・計画在庫を再計算（進度は在庫再計算で実施）
5. POST /line-gantt-plans/generate → ガントを再生成

注意: 前ライン（他ライン）への需要反映は保存時ではなく、**取り込み（pickup）**で行う運用です。

計画削除時の連動削除

計画を削除すると、関連する以下のデータも削除されます：

テーブル	削除条件
LinePlan	直接削除
LineGanttPlan	line_id, plan_date, product_id で削除
LineBacklog	plan_id で削除（展開先の中間品も含む）

重要: LineBacklogは plan_id で削除することで、展開先の製品ID（元の製品IDと異なる場合）も正しく削除されます。

共用部品の処理

異なる完成品（A製品、B製品）が同一ライン内で同じ部品を使用する場合：

- **旧方式:** 同じキー（出力品ID, 工程ID, 日付, 順序）があればスキップ → **数量不足の可能性**
- **新方式:** 同じキーの場合は **加算** → **正確な合計**

例：

A製品 → 部品X を 100個必要

B製品 → 部品X を 50個必要

↓ 加算方式

部品X の計画数 = 150個

計画変更時は、影響範囲のレコードを**削除してから再作成**することで正確性を保ちます。

12. テスト時のデータクリア（私が実演時用に説明しなくていい）

テスト前に以下のテーブルをクリアします：

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE pm_db.t_line_gantt_plan;
TRUNCATE TABLE pm_db.line_plan;
TRUNCATE TABLE pm_db.line_backlog;
```

```
TRUNCATE TABLE pm_db.t_production_order;  
SET FOREIGN_KEY_CHECKS = 1;
```

注意: 外部キー制約があるため、`SET FOREIGN_KEY_CHECKS = 0`で一時的に無効化します。